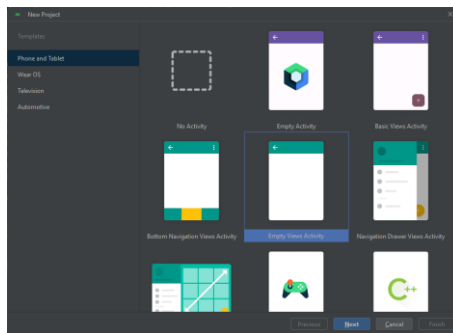


Ejercicio 2 ANDROID - Layouts (1)

En esta actividad vas a crear un nuevo proyecto e insertaremos sobre él varios elementos (Views) desde el modo de “Diseño” de Android Studio.

Primero crea un nuevo proyecto en Android Studio del tipo “Empty Views Activity”.



Llámale “**Layouts1**”, el Lenguaje seleccionado “Kotlin” y aunque tenemos diversas plataformas y APIs en las que utilizar nuestra aplicación, nosotros nos centraremos en aplicaciones para teléfonos y tablets, en cuyo caso tan sólo tendremos que seleccionar la API mínima (es decir, la versión mínima de Android) que soportará la aplicación. Android 9.0. como versión mínima (API 28):

Una vez configurado todo pulsamos el botón **Finish** y Android Studio creará por nosotros toda la estructura del proyecto y los elementos indispensables que debe contener.

1. Concepto de Layouts

A la hora de crear el diseño para nuestras actividades es de gran importancia tener en claro que son los **Layouts en Android**. Un Layout es un elemento que **representa el diseño de la interfaz de usuario de componentes gráficos como una actividad, fragmento o un widget**.

Como sabes, una **Activity** es un componente fundamental en Android que representa **una única pantalla** con una interfaz de usuario. Cada pantalla visible en una aplicación Android es generalmente una Activity, con una interfaz de usuario definida por un **archivo de diseño, una lógica de manejo de Leventos e interacciones con otros componentes** de la aplicación. Un **Archivo de Diseño o Layout** se asocia comúnmente con una Activity para definir la interfaz de usuario que se mostrará en esa pantalla, ya que se refiere a la estructura y disposición de los elementos de la interfaz de usuario en esa pantalla (y están escritos en XML).

Los Layouts se encargan de actuar como **contenedores de Views (vistos en la Actividad 3.1)** para establecer un orden visual, que facilite la comunicación del usuario con la interfaz de la aplicación.

Al crear los **Layouts**, debes referirte a las clases **View** y **ViewGroup**. Un **ViewGroup** es un elemento visual que **contiene a otros views**. Por lo que tienes que usar los métodos apropiados para añadir correctamente los hijos y crear el diseño que deseas.

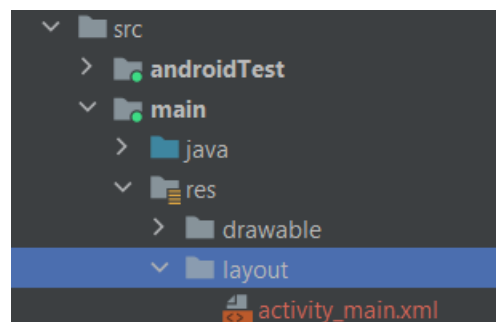
Normalmente se utiliza el estilo declarativo con **XML** para crear los interfaces. No obstante, en ocasiones requeriremos **modificar alguna propiedad** de los Layouts **en tiempo real (dinámica)**, por ejemplo, si deseáramos desaparecer un texto de la interfaz que ya no es necesario, deberíamos acudir al estilo programático (código) para deshabilitar la visibilidad de dicho elemento.

Esto quiere decir, que declarar los elementos de la UI en archivos XML es útil para crear todo el aspecto visual que se usará en la app de forma **estática**.

Pero si los elementos cambian por algún motivo cuando la app está en funcionamiento, entonces se requiere **código Kotlin** para modificar dinámicamente los elementos de la UI. Ambos métodos no son excluyentes.

Las definiciones XML para los layouts se guardan dentro del subdirectorio layout (donde como has visto, se encuentra el archivo activity_main.xml)

:



Cada recurso del tipo layout debe ser un archivo XML, donde el elemento raíz solo puede ser un ViewGroup o un View. Dentro de este elemento puedes incluir **hijos** que definan la estructura del diseño.

Algunos de los **view groups** más populares son:

- **FrameLayout**
- **LinearLayout**
- **RelativeLayout**
- **TableLayout**
- **GridLayout**

a. ATRIBUTOS DE UN LAYOUT

Todas las **características** de un ViewGroup o un View son representadas a través de **atributos** que determinan algún aspecto. Encontrarás atributos para el **tamaño**, la **alineación**, **padding**, **bordes**, **backgrounds**, etc.

Cada atributo XML descrito para un componente tiene una referencia en la clase kotlin que lo representa. Esto quiere decir que al usar un **elemento <TextView>**, nos estamos refiriendo a la **clase TextView**.

Identificador de un view— Existe un atributo que heredan todos los elementos llamado id. Este representa un identificador único para cada elemento. Lo que permite obtener una referencia de cada objeto de forma específica. La sintaxis para el id sería la siguiente:

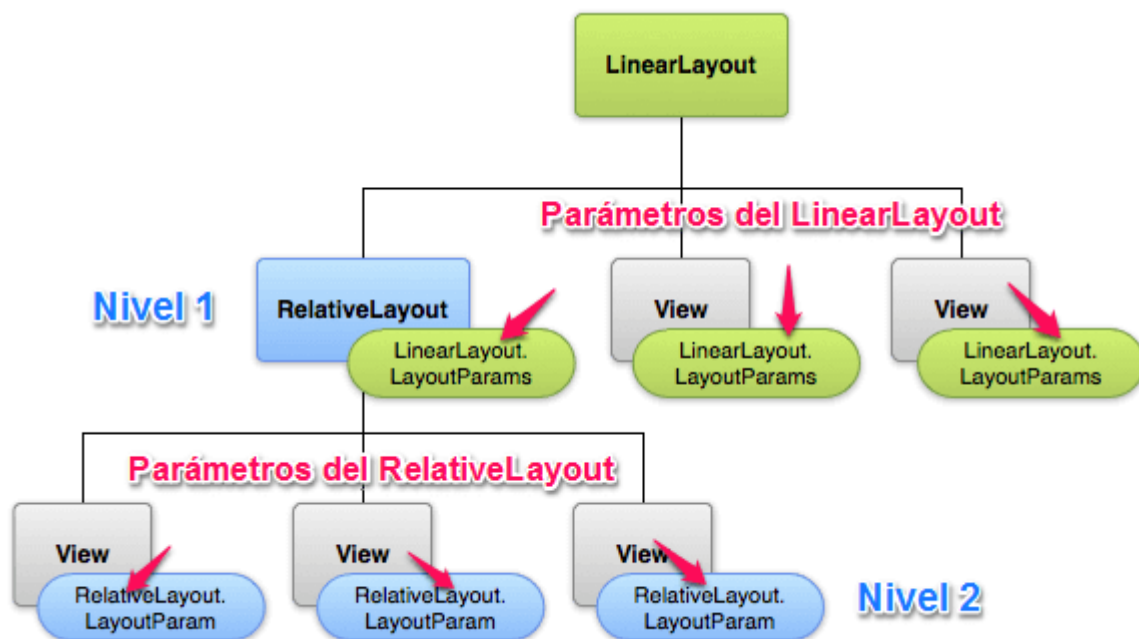
```
android:id="@+id/nombre_identificador"
```

b. PARÁMETROS DE UN LAYOUT

Los parámetros de un layout son atributos especiales que determinan como se **relacionan los hijos de un ViewGroup** dependiendo de su posición y tamaño.

Estos se escriben de la forma `layout_parametro` para someter al view en cuestión. Programáticamente se representan como una clase interna llamada `ViewGroup.LayoutParams`.

Dependiendo de la jerarquía encontrada en el layout, así mismo se aplican los parámetros:



El dibujo anterior, muestra un layout cuyo elemento raíz es un **LinearLayout**. Sus tres hijos del nivel 1 deben ajustarse a los parámetros que este les impone.

Uno de los hijos es un **RelativeLayout**, el cual tiene tres hijos. Como ves, cada elemento del segundo nivel está sometido a los parámetros del RelativeLayout.

Dependiendo del **ViewGroup**, se asignarán los parámetros. Pero existen dos que son comunes independientemente del elemento. Estos son **layout_width** y **layout_height**. que definen el **ancho y la altura** respectivamente de un **cualquier view**:

- Es posible asignarles **medidas absolutas** definidas en dps, sin embargo Google recomienda hacerlo cuando sea estrictamente necesario, ya este tipo de medidas pueden afectar la UI en diferentes tipos de pantalla.

- Como vimos en la Actividad 3.1, hay dos constantes que ajustan automáticamente las dimensiones de un view:
- - o **wrap_content**: Ajusta el tamaño al espacio mínimo que requiere el view.
 - o **match_parent**: Ajusta el tamaño a las dimensiones máximas que el “padre” le permita.

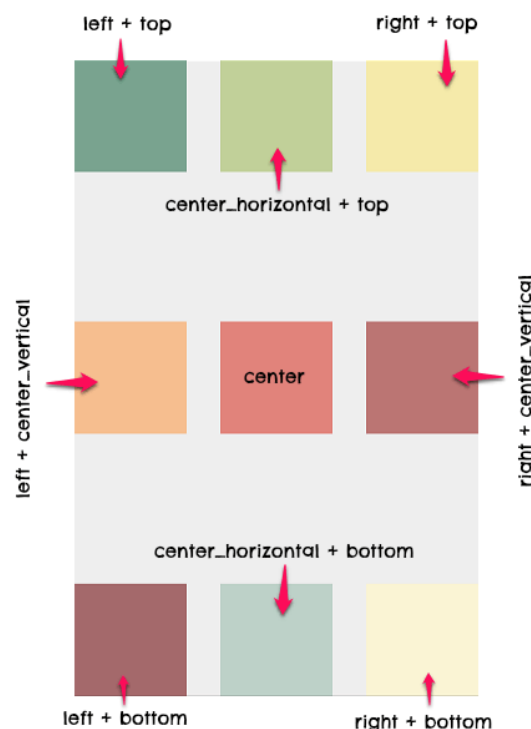


2. Aplicación de Layouts en Android Studio

a. FRAMELAYOUT

Un **FrameLayout** es un view group creado para mostrar **un solo elemento en pantalla o varios elementos dentro del mismo marco** (que se superpondrían en caso de no organizarlos).

Al añadir varias Views (hijos) con el fin de superponerlos, hay que tener en cuenta que el ultimo hijo agregado, es el que se muestra en la parte superior y el resto se ponen por debajo en forma de pila. Para alinear/organizar cada elemento dentro del **FrameLayout**, usa el parámetro **android:layout_gravity** (ten en cuenta las diferencias entre los atributos **gravity** y **layout_gravity**):



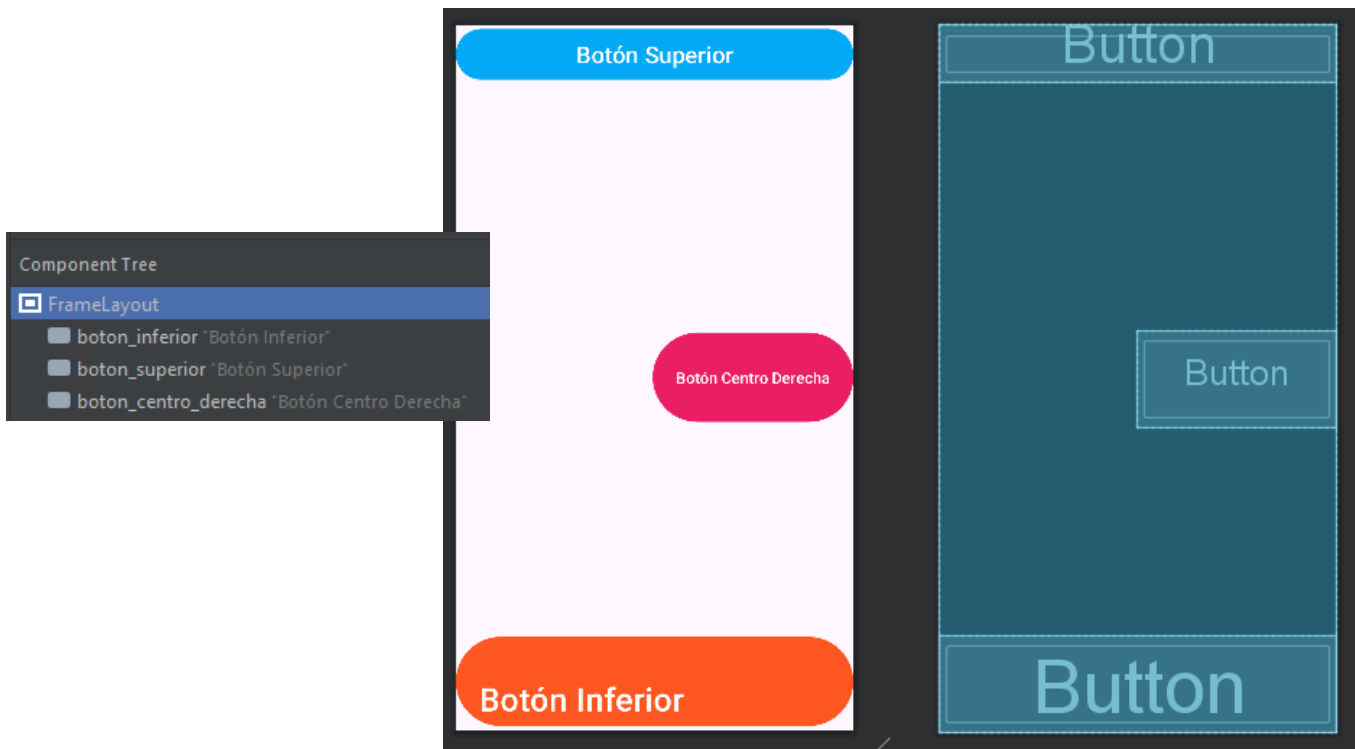
- Crea ahora un **FrameLayout** en el proyecto que has creado:

Para ello, modifica en el modo Diseño (o a través de código) el archivo **activity_main.xml**, transformándolo a tipo “**FrameLayout**” (puedes hacerlo de manera similar a lo visto en la Actividad 3.1).

Después, añade 3 botones y modifícalos para que quede de manera similar a esto:

Atributos a modificar en los **3 botones**:

android:id	android:layout_width	android:layout_height	android:layout_gravity
android:gravity	android:backgroundTint	android:text	android:textSize



Una vez hecho, refactoriza (cambia) el nombre del archivo a: **frame_layout.xml**

Para ello presiona click derecho en archivo, en seguida ve a **Refactor > Rename...** y cambia el nombre. También puedes hacerlo empleando la combinación **SHIFT+F6**.

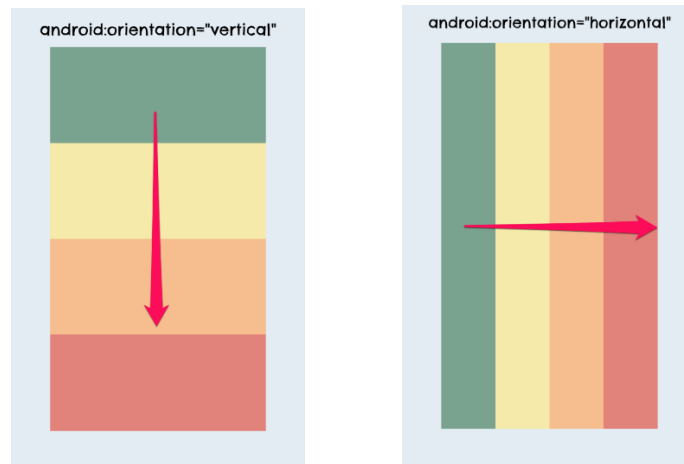
Ejecuta la App en un dispositivo (virtual o físico) y comprueba que está todo correcto.

Guarda el proyecto.

b. LINEARLAYOUT

El **LinearLayout** es un view group que distribuye sus hijos en una sola dimensión establecida. Es decir, o todos organizados en una sola columna (**vertical**) o en una sola fila (**horizontal**). La orientación puedes elegirla a través del atributo **android:orientation**.

A partir de ahí se irán apilando según vayamos añadiendo Views en función de la orientación seleccionada:



El LinearLayout permite asignar un parámetro llamado **android:layout_weight**, el cual define la **importancia** que tiene un view dentro del linear layout. A mayor importancia, más espacio podrá ocupar. El espacio disponible total sería la suma de las alturas (6), por lo que 3 representa el 50%, 2 el 33,33% y 1 el 16,66%.



También es posible definir la **suma total** del espacio con el atributo **android:weightSum**. En función de este valor, los weights serán ajustados:

```
android:weightSum="6"
```

Para **distribuir todos los elementos de igual manera** sobre el espacio total del layout, puedes usar el atributo **height** con valor cero:

```
android:layout_height="0dp"  
android:layout_weight="3"
```

- **Teniendo en cuenta lo visto anteriormente, crea un LinearLayout para que quede similar a este:**

Conectar

Correo

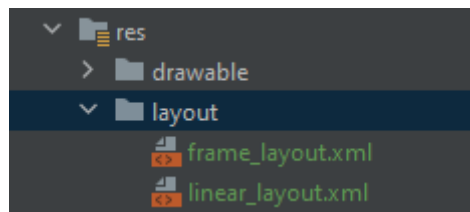
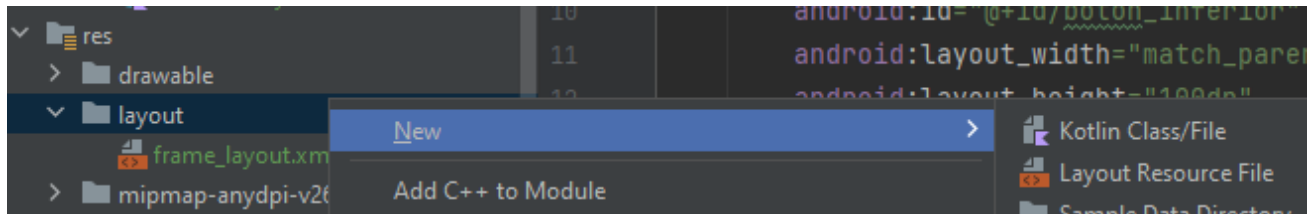
Contraseña

INICIAR SESION

[¿Olvidaste tu contraseña?](#)

Para ello, primero crea un nuevo Layout de nombre: **linear_layout.xml**

layout>New>Layout Resource File



Se compone de **2 TextView**, **2 EditText** (PlainText en el modo Diseño), y **1 Button**.

Atributos que vas a necesitar para **los EditText**:

android:id android:layout_width android:layout_height android:layout_gravity
android:layout_weight android:ems android:hint android:inputType

Una vez acabado, ejecuta la App en un dispositivo (virtual o físico) y comprueba que está todo correcto.

Para ello necesitarás cambiar la ejecución de la Activity configurada en el archivo **MainActivity.kt**, ya que no hemos generado archivos lógicos (.kt) para los 2 layout que has creado:

```
MainActivity.kt x
1  package com.example.layouts
2
3  import ...
4
5
6  class MainActivity : AppCompatActivity() {
7      override fun onCreate(savedInstanceState: Bundle?) {
8          super.onCreate(savedInstanceState)
9          setContentView(R.layout.frame_layout)
10     }
11 }
```

Una vez cambiado este archivo ya puedes depurarlo, y enviarlo a los dispositivos para probarlo.

Guarda el proyecto y sube los archivos a Moodle.